If you're newer to Active Record or haven't worked with it as much, you might be wondering how Ruby generates SQL. Read on to learn more.

Transforming Ruby to SQL

As a Rails developer you may have only created indexes using Active Record Migrations and be less familiar with the SQL statements they generate.

In the last section you saw a CREATE INDEX statement in SQL by creating it with SQL first and then an example of how it's created with Ruby.

You may wish to explore the definitions as creation statements for all of the Indexes when removing and adding back Indexes. PostgreSQL tracks the index creation statements internally in system catalogs. Query the pg_indexes catalog from psql as follows to take a look.

```
SELECT indexdef FROM pg_indexes WHERE tablename = 'users';
```

The statement above shows the index definitions as creation statements for all indexes on the public.users table.

Displayed this way, the creation statements do not contain the CONCURRENTLY keyword even though they may have been created using it.

On your non-live system Rideshare database, practice with more SQL Index modifications. Try dropping an index. Remember that your local development database is a great place to practice. You can always reset the Rideshare schema completely to the original configuration.

From psql, run the DROP INDEX statement below to drop an index named "index _users_on_last_name".

```
DROP INDEX index_users_on_last_name;
```

Dropping Indexes in a live system is also an operation that should use the CONCURRENTLY keyword:

```
DROP INDEX CONCURRENTLY index_users_on_email;
```

If there are other transactions using the index, CONCURRENTLY avoids introducing long-running locks that would block those transactions.

Now that the indexes are dropped, try manually creating them using the SQL statement and the CONCURRENTLY keyword. Make sure that \timing is enabled.